3-데이터-저장-구조

0. 개요

이 문서는 수집한 데이터를 어디에 어떤 형태로 저장할지 정리하는 용도

핵심은 단순함

즉 저장 구조는 단순 보관용이 아니라
취침 전 예측, 기상 후 사후 분석, A(누적 패턴 데이터) 갱신까지 이어질 수 있게 만드는 구조

---

1. 전체 저장 방향

저장은 크게 3층으로 보면 됨

1-1. 로컬 / 운영 저장

기본 저장소는 AWS 상의 운영용 저장 구조와 RPi 로컬 버퍼 구조를 함께 고려

역할 기준으로 보면

- **RPi 로컬**
  - 업로드 전 임시 버퍼
  - 장애 시 최소 데이터 보존
- **AWS 처리 결과 저장**
  - 정규화 데이터
  - 예측 결과
  - 사후 분석 결과
  - A(누적 패턴 데이터)
  - 예측 시점 feature snapshot

즉 핵심 기능은 AWS 기반 처리 구조를 전제로 하되,
RPi에는 업로드 전 버퍼와 장애 대응용 최소 저장을 두는 구조

1-2. 원본 보관

필요하면 원본도 따로 남김

대상

- Fitbit raw JSON
- 센서 raw 값

원본 데이터는 다시 확인할 수 있게 남겨두고,
서비스에서 자주 쓰는 값은 바로 조회하기 좋게 정리해서 저장

1-3. 백업 / 장기 보관

장기 보관이나 백업은 AWS 쪽으로 확장

대상 예시

- raw JSON 백업
- 사후 분석 결과 백업
- A 백업
- 예측 feature snapshot 백업

즉 운영은 AWS 중심, raw/백업은 S3 중심 구조

---

2. 저장 원칙

2-1. 시계열 데이터는 timestamp 기준 저장

심박, steps, calories, 환경 데이터처럼 시간 흐름이 중요한 데이터는 timestamp 기준 저장

이렇게 해야

- 취침 전 1시간 구간 추출
- 수면 중 구간 추출
- Fitbit 데이터와 환경 데이터 시간 정렬
  이 가능

2-2. 일 단위 요약 데이터는 date 기준 저장

하루 단위로 보는 값은 date 기준 저장

예시

- 안정시 심박수
- 수면 요약

2-3. raw와 정규화 데이터를 분리 저장

원본은 원본대로 남기고,
서비스가 자주 쓰는 값은 정규화해서 저장

이유

- 원본 재확인 가능
- 서비스 조회 단순화
- 분석 로직 단순화

2-4. 내부 생성 데이터도 저장

이 서비스는 입력 데이터만 저장하면 안 되고,
시스템이 만든 결과도 저장해야 함

대상

- Sleep Score
- 취침 전 예측 결과
- 기상 후 사후 분석 결과
- A(누적 패턴 데이터)
- 예측 시점 요약 feature snapshot

그래야

- 예측이 실제로 맞았는지 비교 가능
- 다음 판단에 다시 활용 가능
- 설명 문구 생성 근거 유지 가능

2-5. 자주 쓰는 feature는 예측 시점 snapshot으로 저장

원본 시계열만 보관하는 것이 아니라,
예측 당시 실제로 사용한 요약값도 함께 저장

이유

- 그 시점에 무엇을 보고 예측했는지 복원 가능
- 예측 결과와 입력값 비교가 쉬움
- 이후 rule-based 조건 정리에도 유리

---

3. 저장 대상별 구조

3-1. Fitbit 데이터

3-1-1. 심박 시계열

저장 이유

- 취침 전 상태 확인
- 수면 중 심박 흐름 분석
- 환경 데이터와 결합
- 사후 분석 활용

저장 형태

- 분 단위 시계열 데이터
- timestamp 기준 저장

테이블 예시

`fitbit_heart_intraday`

주요 컬럼

- id
- user_id
- timestamp
- heart_rate
- unit
- source_date
- created_at

우리 저장 예시

```
{
  "user_id": "user-01",
  "timestamp": "2026-04-10T22:31:00+09:00",
  "heart_rate": 78,
  "unit": "bpm",
  "source_date": "2026-04-10",
  "created_at": "2026-04-10T22:31:05+09:00"
}
```

3-1-2. 안정시 심박수

저장 이유

- 사용자 기준선
- 현재 심박과 비교
- A 갱신에 활용

저장 형태

- 일 단위 요약값

테이블 예시

`fitbit_resting_heart_rate_daily`

주요 컬럼

- id
- user_id
- date
- resting_heart_rate
- unit
- created_at

우리 저장 예시

```
{
  "user_id": "user-01",
  "date": "2026-04-10",
  "resting_heart_rate": 64,
  "unit": "bpm",
  "created_at": "2026-04-10T08:10:00+09:00"
}
```

3-1-3. 걸음 수 시계열

저장 이유

- 취침 전 활동량 확인
- 늦은 시간 활동 여부 확인
- 예측 입력값 활용

저장 형태

- 분 단위 시계열 데이터
- timestamp 기준 저장

테이블 예시

`fitbit_steps_intraday`

주요 컬럼

- id
- user_id
- timestamp
- steps
- source_date
- created_at

우리 저장 예시

```
{
  "user_id": "user-01",
  "timestamp": "2026-04-10T22:31:00+09:00",
  "steps": 12,
  "source_date": "2026-04-10",
  "created_at": "2026-04-10T22:31:05+09:00"
}
```

3-1-4. 칼로리 시계열

저장 이유

- 활동 강도 확인
- steps 보완
- 취침 전 상태 해석

저장 형태

- 분 단위 시계열 데이터
- timestamp 기준 저장

테이블 예시

`fitbit_calories_intraday`

주요 컬럼

- id
- user_id

- timestamp
- calories
- activity_level
- mets
- source_date
- created_at

우리 저장 예시

```
{
  "user_id": "user-01",
  "timestamp": "2026-04-10T22:31:00+09:00",
  "calories": 1.3,
  "activity_level": 0,
  "mets": 10,
  "source_date": "2026-04-10",
  "created_at": "2026-04-10T22:31:05+09:00"
}
```

3-1-5. 수면 요약

저장 이유

- 실제 수면 결과 확인
- 수면 시작/종료 시각 확인
- 사후 분석 기준 데이터
- Sleep Score 계산 입력값

저장 형태

- 일 단위 요약 데이터

테이블 예시

`fitbit_sleep_summary`

주요 컬럼

- id
- user_id
- date
- start_time

- end_time
- duration_ms
- minutes_asleep
- minutes_awake
- time_in_bed
- is_main_sleep
- created_at

우리 저장 예시

```
{
  "user_id": "user-01",
  "date": "2026-04-11",
  "start_time": "2026-04-10T23:52:00+09:00",
  "end_time": "2026-04-11T07:18:00+09:00",
  "duration_ms": 26760000,
  "minutes_asleep": 402,
  "minutes_awake": 24,
  "time_in_bed": 446,
  "is_main_sleep": true,
  "created_at": "2026-04-11T08:05:00+09:00"
}
```

3-1-6. 수면 단계

저장 이유

- 실제 수면 질 세부 흐름 확인
- Deep & REM 점수 계산
- 이전 N일 수면 단계 패턴 누적
- 사후 분석 활용

저장 형태

- 구간 단위 시계열 데이터
- timestamp 기준 저장

테이블 예시

`fitbit_sleep_levels`

주요 컬럼

- id
- user_id
- timestamp
- sleep_level
- duration_seconds
- source_date
- created_at

우리 저장 예시

```
{
  "user_id": "user-01",
  "timestamp": "2026-04-11T00:12:00+09:00",
  "sleep_level": "light",
  "duration_seconds": 1800,
  "source_date": "2026-04-11",
  "created_at": "2026-04-11T08:05:00+09:00"
}
```

3-1-7. Fitbit 원본 응답

저장 이유

- 디버깅
- 재처리
- 응답 구조 변경 대응

저장 형태

- raw JSON 보관

테이블 또는 파일 예시

- `fitbit_raw_archive`
- 또는 날짜별 JSON 파일

주요 컬럼 예시

- id
- user_id
- data_type
- source_date

- raw_json
- fetched_at

우리 저장 예시

```
{
  "user_id": "user-01",
  "data_type": "heart_intraday",
  "source_date": "2026-04-10",
  "raw_json": "{ ...fitbit raw response... }",
  "fetched_at": "2026-04-10T22:31:05+09:00"
}
```

3-2. 환경 데이터

환경 데이터는 센서 모델이 확정된 현재 기준으로,
서비스에서 같이 조회하기 쉽게 한 timestamp 기준으로 묶어서 저장하는 구조

수집 대상

- 온도
- 습도
- 실내 가스 변화 지표
- 가스/공기질 지표값

3-2-1. 환경 정규화 데이터

저장 이유

- 취침 전 1시간 환경 상태 조회
- 수면 중 환경 변화 분석
- 예측과 사후 분석에서 한 번에 사용 가능

저장 형태

- 1분 단위 시계열 데이터
- 한 행에 온도 / 습도 / 가스 지표값 / 공기질 지표값 같이 저장

테이블 예시

`environment_data`

주요 컬럼

- id
- user_id
- timestamp
- temperature
- temperature_unit
- humidity
- humidity_unit
- gas_index
- gas_index_unit
- air_quality_index
- air_quality_unit
- created_at

우리 저장 예시

```
{
  "user_id": "user-01",
  "timestamp": "2026-04-10T22:31:00+09:00",
  "temperature": 24.8,
  "temperature_unit": "C",
  "humidity": 58.2,
  "humidity_unit": "%",
  "gas_index": 0.62,
  "gas_index_unit": "normalized_index",
  "air_quality_index": 0.42,
  "air_quality_unit": "sensor_index",
  "created_at": "2026-04-10T22:31:01+09:00"
}
```

이 구조로 가면

- 취침 전 1시간 환경 상태 조회
- 같은 시각의 환경값 묶음 확인
  이 쉬워짐

3-2-2. 환경 raw 데이터

저장 이유

- 센서 raw 값 재확인
- 전처리 변경 대응
- 이상치 추적

- MQ-5 raw 값 기반 재처리 대응

저장 형태

- 센서별 원본값 보관

테이블 예시

`environment_raw`

주요 컬럼

- id
- user_id
- sensor_type
- device_id
- timestamp
- raw_value
- unit
- quality
- created_at

우리 저장 예시

```
{
  "user_id": "user-01",
  "sensor_type": "mq5_gas_sensor",
  "device_id": "sensor-01",
  "timestamp": "2026-04-10T22:31:00+09:00",
  "raw_value": 372,
  "unit": "raw",
  "quality": "ok",
  "created_at": "2026-04-10T22:31:01+09:00"
}
```

3-3. 사용자 입력 데이터

3-3-1. 주관 만족도 점수

저장 이유

- 객관 데이터만으로 안 보이는 체감 품질 반영
- 개인화 기준 형성

- 사후 분석 보정

저장 형태

- 기상 후 1회 입력
- 0~100 점수제

테이블 예시

`user_input_sleep_satisfaction`

주요 컬럼

- id
- user_id
- timestamp
- score
- scale
- created_at

우리 저장 예시

```
{
  "user_id": "user-01",
  "timestamp": "2026-04-11T08:20:00+09:00",
  "score": 63,
  "scale": "0-100",
  "created_at": "2026-04-11T08:20:00+09:00"
}
```

3-4. 내부 생성 데이터

3-4-1. Sleep Score

저장 이유

- 실제 수면 결과를 대표 점수 1개로 요약
- 사용자에게 보여줄 기준 점수 확보
- 사용자 체감과의 차이를 추적
- A 갱신에 활용

저장 형태

- 기상 후 수면 로그 확보 후 1회 계산

테이블 예시

`sleep_score_result`

주요 컬럼

- id
- user_id
- target_date
- total_score
- time_asleep_score
- deep_rem_score
- restoration_score
- created_at

예시

```
{
  "user_id": "user-01",
  "target_date": "2026-04-11",
  "total_score": 72,
  "time_asleep_score": 38,
  "deep_rem_score": 17,
  "restoration_score": 17,
  "created_at": "2026-04-11T08:06:00+09:00"
}
```

3-4-2. 취침 전 예측 결과

저장 이유

- 오늘 수면 질 예측 결과 기록
- 실제 결과와 비교 가능
- 예측 품질 점검 가능

저장 형태

- 취침 전 예측 1회마다 1행 저장

테이블 예시

`prediction_result`

주요 컬럼

- id
- user_id
- timestamp
- risk_score
- risk_level
- recommendation_level
- recommendation_text
- model_mode
- created_at

설명

- risk_score: 수면 질 저하 가능성 점수
- risk_level: 낮음 / 보통 / 높음 같은 내부 수준
- recommendation_level: 권장 / 주의 / 경고
- recommendation_text: 사용자에게 보여줄 문구
- model_mode: slm_assisted / rule_based / hybrid 같은 예측 방식 구분

우리 저장 예시

```
{
  "user_id": "user-01",
  "timestamp": "2026-04-10T23:00:00+09:00",
  "risk_score": 0.74,
  "risk_level": "high",
  "recommendation_level": "warning",
  "recommendation_text": "실내 가스 변화 지표가 높아요. 잠들기 전에 환기를 하면 좋을
것 같아요.",
  "model_mode": "hybrid",
  "created_at": "2026-04-10T23:00:01+09:00"
}
```

3-4-3. 기상 후 사후 분석 결과

저장 이유

- 실제 결과 해석 기록
- 다음 예측 기준 확보
- 설명 문구 생성 근거 유지

저장 형태

- 기상 후 분석 1회마다 1행 저장

테이블 예시

`analysis_result`

주요 컬럼

- id
- user_id
- target_date
- summary_text
- factor_json
- score_gap_note
- created_at

설명

- summary_text: 사용자에게 바로 보여줄 수 있는 한 줄 설명
- factor_json: 구조화된 원인 정보
- score_gap_note: 자동 점수와 체감 차이 설명

예시

```
{
  "user_id": "user-01",
  "target_date": "2026-04-11",
  "summary_text": "어젯밤 수면 질 점수는 72점이었어요. 주요 원인은 높은 실내 가스 지표였고, 보조 원인은 취침 전 높은 심박이었을 가능성이 있어요.",
  "factor_json": {
    "main_factor": "high_gas_index",
    "sub_factor": "high_pre_sleep_heart_rate"
  },
  "score_gap_note": "데이터상으로는 수면 흐름이 비교적 안정적이었지만, 사용자는 실제로는 수면 질이 더 낮게 느껴졌어요.",
  "created_at": "2026-04-11T08:25:00+09:00"
}
```

3-4-4. A(누적 패턴 데이터)

저장 이유

- 다음 예측 기준으로 재사용
- 개인화 보정의 핵심 데이터

저장 형태

- 기상 후 분석 후 갱신

테이블 예시

`pattern_profile`

주요 컬럼

- id
- user_id
- updated_at
- profile_json
- version

예시

```
{
  "user_id": "user-01",
  "updated_at": "2026-04-11T08:30:00+09:00",
  "profile_json": {
    "avg_sleep_minutes_recent": 392,
    "avg_satisfaction_recent": 74,
    "avg_pre_sleep_heart_rate": 81,
    "avg_wake_heart_rate": 76,
    "gas_sensitivity": "moderate",
    "score_gap_trend": "automatic_score_higher_than_subjective"
  },
  "version": 3
}
```

3-4-5. 예측 시점 feature snapshot

저장 이유

- 그 시점에 무엇을 보고 예측했는지 복원 가능
- 예측 결과와 입력값 비교가 쉬움
- 이후 rule-based 기준 정리에도 유리

저장 형태

- 취침 전 예측 시 1회 저장

테이블 예시

`prediction_feature_snapshot`

주요 컬럼

- id
- user_id
- timestamp
- feature_json
- created_at

예시

```
{
  "user_id": "user-01",
  "timestamp": "2026-04-10T23:00:00+09:00",
  "feature_json": {
    "avg_heart_rate_1h": 81.4,
    "max_heart_rate_1h": 96,
    "steps_sum_1h": 214,
    "calories_sum_1h": 58.2,
    "avg_temp_1h": 24.6,
    "avg_humidity_1h": 57.8,
    "avg_gas_index_1h": 0.39,
    "max_gas_raw_1h": 945,
    "recent_n_sleep_avg": 401
  },
  "created_at": "2026-04-10T23:00:01+09:00"
}
```

---

4. 저장 구조 핵심 정리

이 문서에서 중요한 것은
무엇을 저장하느냐보다,
왜 그 저장이 다음 예측과 분석으로 이어지는가를 보는 것

즉 이 서비스의 저장 구조는

- 수집 데이터 저장

- 내부 생성 데이터 저장
- 실제 결과 저장
- 체감 평가 저장
- A(누적 패턴 데이터) 갱신
- 다음 예측 반영

이 순환 구조를 가능하게 하는 저장 구조